Introduzione all'astrazione

Python

L'astrazione è un concetto fondamentale nella programmazione e nel paradigma della programmazione orientata agli oggetti (OOP).

Nella sua essenza, l'astrazione è il processo di nascondere i dettagli complessi di un sistema, esponendo solo le operazioni rilevanti per l'utente.

In Python, questo è spesso realizzato attraverso l'uso di classi che definiscono metodi e attributi, permettendo agli sviluppatori di utilizzare oggetti senza necessariamente comprendere il funzionamento interno di ogni componente.

Questo permette di concentrarsi su una programmazione a più alto livello, rendendo il codice più leggibile, manutenibile e riutilizzabile.

L'astrazione è una delle caratteristiche chiave della programmazione orientata agli oggetti (OOP) e presenta diverse caratteristiche importanti che ne definiscono il ruolo e l'utilità nello sviluppo del software:

Semplificazione:
L'astrazione aiuta a ridurre la complessità del programma permettendo agli sviluppatori di concentrarsi su un livello di interazione più alto. Ciò si ottiene nascondendo i dettagli tecnici più complessi dietro un'interfaccia semplice da usare.

Riduzione di dipendenze:
Attraverso l'uso di astrazioni, i componenti di un sistema possono essere resi meno dipendenti l'uno dall'altro. Questo perché le interazioni avvengono tramite interfacce ben definite piuttosto che attraverso componenti specifici, facilitando così la manutenzione e l'aggiornamento del sistema.

Riusabilità:
Le classi astratte possono essere utilizzate come basi per altre classi.
Questo permette agli sviluppatori di riutilizzare codice comune e di estenderlo dove necessario, senza riscrivere il codice esistente.

Modularità:
L'astrazione favorisce una progettazione più modulare del software. I componenti possono essere sviluppati e testati in maniera indipendente gli uni dagli altri, il che migliora la gestione del progetto e l'isolamento dei problemi.

Incapsulamento:
Sebbene strettamente legato all'astrazione, l'incapsulamento si concentra sul nascondere i dettagli interni del funzionamento di una classe e sull'esporre solo quei metodi e proprietà che sono necessari per l'uso della classe. L'astrazione pensa più a ciò che un oggetto fa piuttosto che a come lo fa, mentre l'incapsulamento nasconde il "come".

Interfacciamento:
Le interfacce e le classi astratte sono due strumenti che Python e altri linguaggi OOP utilizzano per implementare l'astrazione.
Consentono di definire schemi e funzionalità che le classi concrete devono poi implementare, garantendo un certo livello di uniformità nel modo in cui le diverse parti del software interagiscono tra loro.

L'astrazione, quindi, non solo facilita la gestione della complessità nei progetti di sviluppo software, ma supporta anche l'efficienza e la flessibilità nella scrittura e nella manutenzione del codice.

In Python, il concetto di astrazione è spesso implementato attraverso il modulo abc che sta per "Abstract Base Classes".

Questo modulo permette agli sviluppatori di definire classi astratte che non possono essere istanziate direttamente, ma servono piuttosto come basi per altre classi.

Le classi astratte possono definire metodi astratti che devono essere implementati dalle sottoclassi.

Abstract Base Classes (ABC)

Le Abstract Base Classes offrono un modo per definire una struttura per le classi senza implementarne completamente il comportamento.

Questo serve a garantire che certi metodi essenziali siano implementati in tutte le classi derivate, mantenendo una certa coerenza e prevedibilità nel design del software.

Le classi Abstract Base Classes (ABC) in Python forniscono una struttura fondamentale per definire le interfacce in modo formale e sono utilizzate per imporre che determinate metodologie di programmazione siano adottate nelle sottoclassi. Alcuni elementi comuni di una classe ABC includono:

Metodi astratti:
Questi metodi sono dichiarati ma non implementati nella classe astratta. L'implementazione deve essere fornita dalle sottoclassi. Se una sottoclasse non implementa un metodo astratto, non sarà possibile istanziarla.

Metodi concreti:
Le classi ABC possono anche avere metodi con implementazioni complete, che possono essere ereditati dalle sottoclassi senza la necessità di sovrascriverli. Questi metodi concreti possono utilizzare altri metodi astratti definiti nella stessa classe.

Proprietà:
Le classi ABC possono definire proprietà che devono essere implementate dalle sottoclassi, utilizzando i getter e setter come metodi astratti.

Inizializzatore:
Anche se non è possibile istanziare direttamente una classe astratta, può contenere un inizializzatore (__init__) che può essere chiamato dalle sottoclassi per inizializzare la parte di classe base.

Metodi di classe e statici:
Allo stesso modo dei metodi concreti, le classi ABC possono includere metodi di classe (@classmethod) e metodi statici (@staticmethod), che possono essere chiamati senza che la classe sia istanziata.

Metodi astratti ovvero @abstractmethod

Un metodo astratto è un metodo che viene dichiarato, ma non implementato, nella classe astratta.

I metodi astratti devono essere implementati dalle sottoclassi.

Se una sottoclasse non implementa tutti i metodi astratti, non può essere istanziata, e tentare di farlo solleverà un errore di tipo TypeError.

Implementazione in Python

Per creare una classe astratta in Python utilizzando il modulo abc, è necessario seguire questi passaggi:

1. **Importare ABC e abstractmethod dal modulo abc.**
2. **Creare una classe che eredita da ABC.**
3. **Decorare i metodi che devono essere astratti con il decoratore @abstractmethod.**

Ecco un esempio pratico:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        """Metodo astratto che deve essere implementato dalla sottoclasse."""
        pass

class Dog(Animal):
    def make_sound(self):
        print("Bark")

# Questo funzionerà
dog = Dog()
dog.make_sound()

# Questo solleverà un errore perché non si può istanziare una classe astratta
# animal = Animal()
```

In questo esempio, Animal è una classe astratta che non può essere istanziata da sola e richiede che il metodo make_sound() sia definito in una sottoclasse.

La classe Dog eredita da Animal e implementa il metodo astratto, quindi può essere istanziata e utilizzata normalmente.

L'uso di ABC e metodi astratti in Python è un potente strumento per garantire che un certo interface contract sia rispettato, fornendo una guida chiara agli sviluppatori su come devono essere strutturate e implementate le classi derivate.

Buon Davante a tutti